



WeCloudData

Designing Tool Systems

Introduction





WeCloudData

How an Agent Uses a Tool? Designing Tool Systems



Why Do Tool Systems Matter?

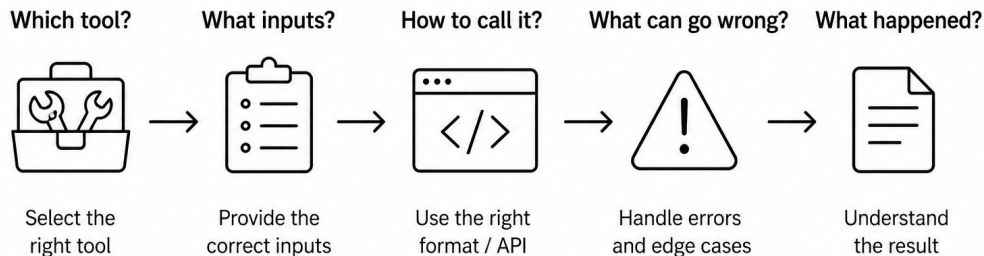
Tools

An agent may need to:

- Search information
- Query databases
- Send emails
- Create calendar events
- Execute code
- Call APIs

Simply giving an agent access to tools isn't enough.

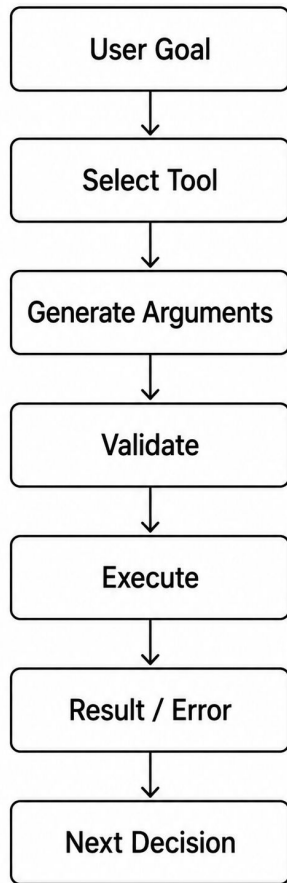
The agent needs to know:





One Tool Call Lifecycle

Tools



Tool calling is a process, not just an API call.



Example: Scheduling a Meeting

Tools

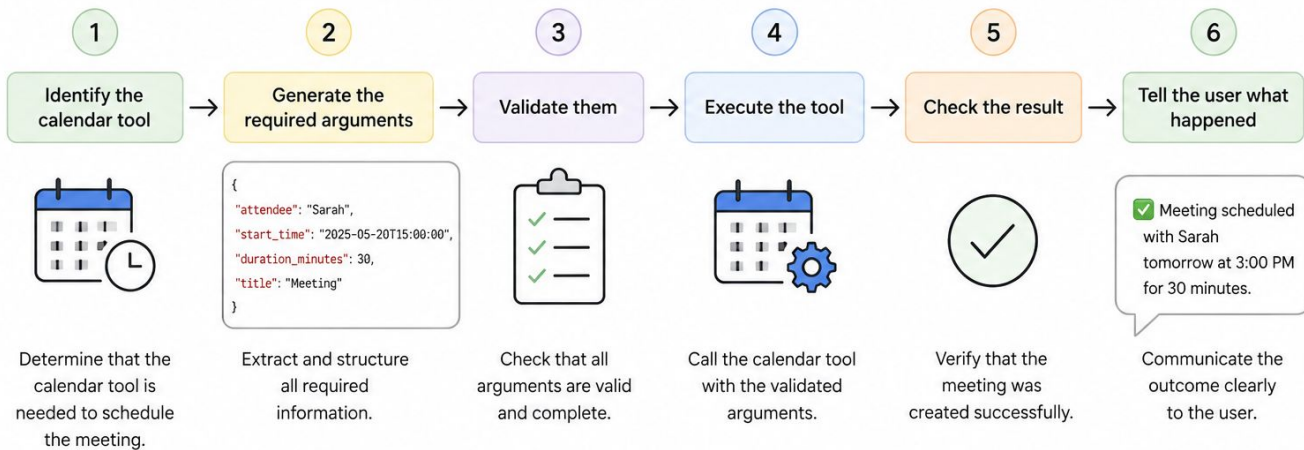


User

"Schedule a meeting with Sarah tomorrow at 3 PM for 30 minutes."



The agent needs to:





Poor Tools Vs Good Tools

Tools

Poor Tool

`calendar()`

What does it do?

- Create?
- Delete?
- Search?
- Update?

The agent has little guidance.

Better Tool

`create_calendar_event()`

Now the intended action is clear.

Clear tool design reduces
ambiguity for the model.



What makes a Good Tool?

Tools

A good tool should have:

- Clear name
- Clear purpose
- Clear usage guidance
- Structured inputs
- Defined output
- Known errors
- Defined permissions
- Observable execution



WeCloudData

Designing a Clear Tool Contract

Designing Tool Systems



What Is a Tool Contract?

Tool Contract

A tool contract defines the agreement between:

Agent ↔ Tool

It Specifies:

What can I do? What inputs do I need? What will I return?

What can go wrong? What am I allowed to do?



Start With One Responsibility

Tool Contract

Too Broad

`manage_calendar()`

Could mean:

- Create event
- Update event
- Delete event
- Search events

Focused

`create_calendar_event()`

Principle

One tool → One clear responsibility



Name Tools Clearly

Tool Contract

A good name should be:

- Descriptive
- Action-oriented
- Specific

Why?

The agent uses the name as part of its decision-making.

Bad Examples

- `calendar()`
- `event()`
- `do_calendar()`

Good Examples

- `create_calendar_event()`
- `update_calendar_event()`
- `delete_calendar_event()`



Explain What the Tool Does

Tool Contract

Tool:

`create_calendar_event()`

Description:

Creates a new calendar event with a specified title, start time, end time, and optional attendees.

Good description answers:

What does this tool actually do?



Explain When to Use It

Tool Contract

The description should also tell the agent **when the tool is appropriate**.

Use when:

- User wants to create a new event
- Required scheduling information is available

Don't use when:

- User wants to search existing events
- User wants to delete an event
- User is only asking for scheduling advice

Why?

A clear tool description helps the agent **select the right tool**.



Define the Inputs

Tool Contract

Each parameter should define:

- Name
- Type
- Required/optional
- Allowed values
- Constraints

Example:

```
create_calendar_event(title, start_time,  
                        end_time, attendees)
```



Input Constraints

Tool Contract

Why?

The agent needs to know **what valid input looks like**.

Example:

title:

type: string

required: yes

start_time:

type: datetime

required: yes



Define the Output

Tool Contract

Don't stop at defining inputs.

Ask: **What does the tool return after execution?**

Why structured output?

The agent can use the result for its **next decision**.

Success:

```
{  
  
  "event_id": "evt_123",  
  "status": "created",  
  
  "start_time": "...",  
  "end_time": "..."  
}
```



Define Possible Errors

Tool Contract

Tools can fail.

The agent needs to know:

What went wrong?

Example:

- INVALID_TIME
- MISSING_FIELD
- CONFLICT
- PERMISSION_DENIED
- CALENDAR_UNAVAILABLE



Not All Errors Are the Same

Tool Contract

Correctable

Missing end time



Ask / infer required
information



Retry

Not immediately correctable

Permission denied



Ask for authorization
/ human help

Stop

Repeated tool
failure



Stop



Inform user

An error contract should tell the agent what happened and what it can do next.



Permissions & Approval

Tool Contract

Some tools can cause real-world consequences.

Examples:

- Send email
- Delete files
- Make purchases
- Modify records
- Book appointments

Does this action require permission or human approval?



Complete Contract

Tool Contract

create_calendar_event()		
	Purpose	Create a new calendar event.
	Inputs	• title — required • start_time — required • end_time — required • attendees — optional
	Output	Event ID + status + event details
	Errors	• Missing field • Invalid time • Conflict • Permission denied
	Approval	Booking may require user confirmation.



Tool Design Anti-Patterns

Tool Contract

Vague Names

`do_calendar()`

Too many Responsibilities

`manage_everything()`

Unclear Parameters

Data

No Constraints

"Provide a time."

No output Definition

"What happens after
execution?"

No Error Contract

"What if it fails?"



Fix the Bad Tool

Tool Contract

With:

- Clear purpose
- Defined inputs
- Defined output
- Defined errors
- Permission rules

Good tool design makes the correct action easier for the agent to choose.

Before

```
manage_calendar(data)
```

After

```
create_calendar_event(  
    title: string,  
    start_time: datetime,  
    end_time: datetime,  
    attendees: list[string])
```



WeCloudData

Structured Inputs, Validation & Recovery **Designing Tool Systems**



Why Validate Tool Arguments?

Structured Tools

The model may generate:



```
start_time = "tomorrow afternoon-ish"
```

or:



```
end_time = "3pm"
```

or:



```
title
```

Even if the model understands the task, the tool still needs **valid inputs**.



Three Layers of Validation

Structured Tools

Structured generation

Is the argument in the expected format?

Schema validation

Are the types and required fields correct?

Semantic/business validation

Does the request actually make sense and is it allowed?



Structured Arguments

Structured Tools

Instead of generating free-form text:

"Schedule Sarah tomorrow at
three..."

Benefit

The tool receives **machine-readable arguments**.

Generate Structured Arguments

```
{  
  "title": "Meeting with Sarah",  
  "start_time":  
    "2026-09-01T15:00",  
  "end_time":  
    "2026-09-01T15:30",  
  "attendees":  
    ["sarah@example.com"]  
}
```



Schema Validation

Tools Validation

A schema can check:

- Required fields
- Data types
- Allowed values
- Formatting

If something is invalid:

Validation error

Example

```
start_time → datetime  
end_time → datetime  
title → string
```



Schema $\neq$ Correctness

Tools Validation

A request can be structurally valid but still wrong.

Both are valid datetimes.

But the request is **semantically invalid**.

```
start_time = 3 PM
```

```
end_time   = 2 PM
```

Therefore:

Valid format $\neq$ valid request



Semantic & Business Validation

Tools Validation

Check rules such as:

- End time must be after start time
- Event must not violate scheduling rules
- User must have permission
- Required resources must be available
- Action must stay within policy

Example

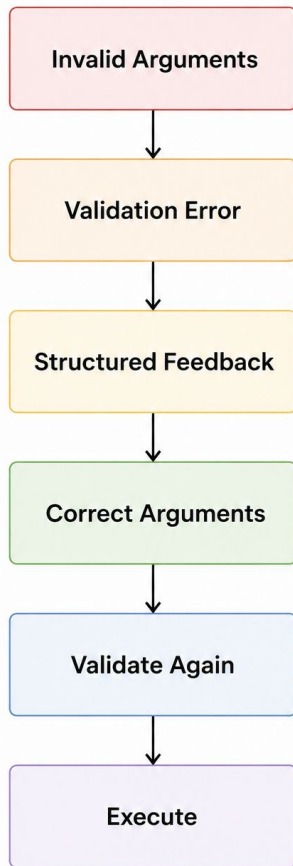
start = 3 PM
end = 2 PM
Schema ✓
Business rule ✗



Recovery Flow

Recovery Flow

When Validation
Fails:



Example

Error:

`"end_time must be
after start_time"`

Agent corrects the
arguments.



When Should the Agent Retry?

Tools Validation

Retry

When the error is:

- Temporary
- Correctable
- Caused by invalid arguments

Ask Human

When:

- Information is missing
- Permission is required
- User intent is ambiguous

Stop

When:

- Repeated failures occur
- Action is unsafe
- Tool is unavailable



WeCloudData

From One Tool to a Tool System

Designing Tool Systems



One Tool Is Not a Tool System

Tool System

An agent may have:

Search
Calendar
Email
Database
Calculator

Now the system needs to answer:

**Which tool is
available?**

**Which one should
be called?**

**How is the call
routed?**

**What happens
after execution?**



Tool Registry

Tool System

What?

A **tool registry** stores available tools and their definitions.

Tool Registry

- |— search_web
- |— create_calendar_event
- |— send_email
- |— query_database
- |— calculate

Why?

The agent needs a discoverable set of available capabilities.

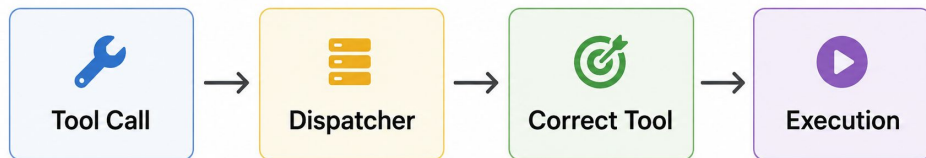


Tool Dispatcher

Tool System

What?

The **dispatcher** receives a validated tool call and sends it to the correct implementation.



Example

"create_calendar_event"
↓
Dispatcher
↓
Calendar Implementation



Orchestrator

Tool System

What?

The **orchestrator** manages the overall execution.

It may handle:

- Sequencing
- Branching
- Parallel execution
- State
- Retries
- Approvals

Difference

**Dispatcher = Where
does this tool call go?**

**Orchestrator = What
happens across the
whole task?**



Sequential Execution

Tool System

Some steps depend on previous results.

Search Availability



Select Time



Create Event

When to Use?

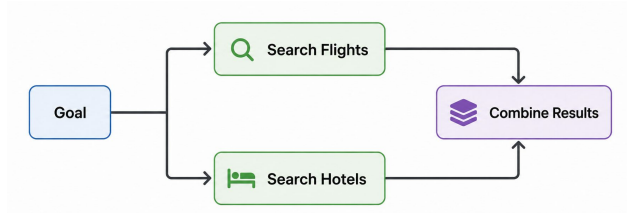
Step B needs the result of
Step A.



Parallel Execution

Tool System

Some actions are independent.



When to Use?

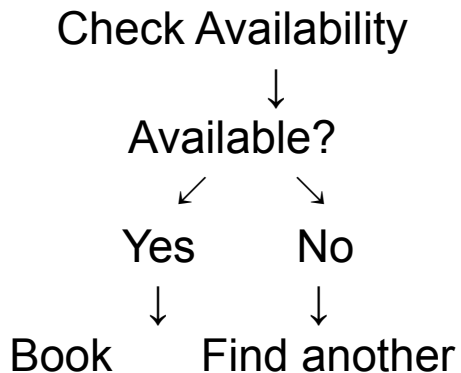
Tasks can run independently.



Conditional Execution

Tool System

The next action can depend on the result.



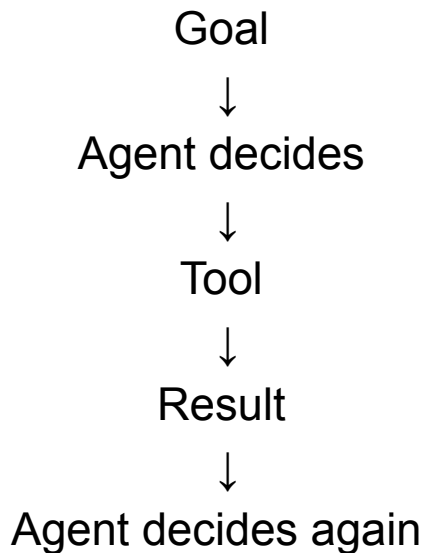
Tool results can determine the next step.



Agent-Directed Execution

Tool System

Sometimes the next step cannot be fully predefined.



The agent dynamically selects the next action based on context and results.



WeCloudData

Testing & Production Readiness Designing Tool Systems



What should we test?

Testing and Production



Don't only test:

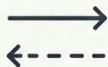
"Does the tool work?"



Test the **agent + tool interaction**.



Agent



Tool



Test:

- ✓ Correct tool selection
- ✓ Valid arguments
- ✓ Invalid arguments
- ✓ Missing arguments
- ✓ Ambiguous requests
- ✓ Expected outputs
- ✓ Known failures
- ✓ Retry behavior
- ✓ Permissions
- ✓ Approval requirements



Test Tool Selection

Testing and Production

User: "Schedule a meeting with Sarah."

Expected:

create_calendar_event

Not:

- search_calendar
- delete_calendar_event
- send_email

Question:

Did the agent choose the right tool?



Test Arguments

Testing and Production

Test different cases:

Valid

title
start_time
end_time

Missing

title

Invalid

end_time
<
start_time

Ambiguous

"Schedule it
tomorrow
afternoon."

Why?

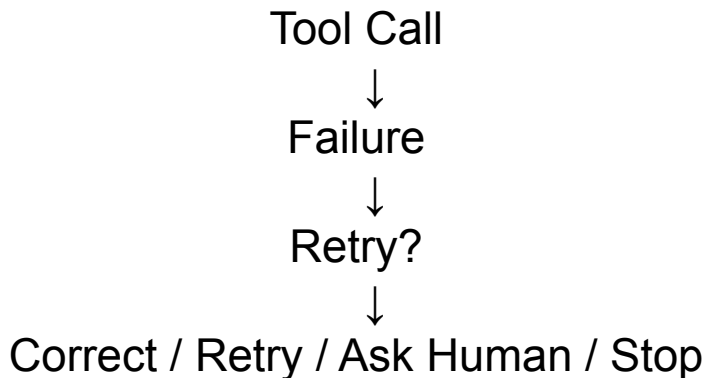
Real users don't always provide perfect inputs.



Test Failure & Recovery

Testing and Production

Test:



Example

Calendar API temporarily unavailable:

Does the agent retry?

Or:

Does it incorrectly claim the event was created?



What Should We Monitor?

Testing and Production

Track tool-specific metrics:

- Tool selection accuracy
- Argument validation failures
- Execution failure rate
- Retry rate
- Latency
- Cost
- Approval rate

Why?

You need to know:

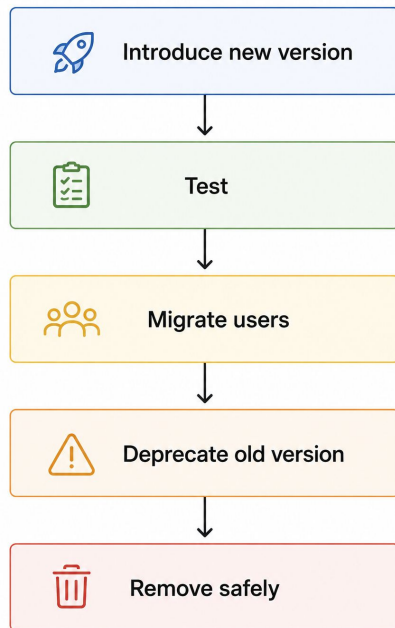
Where is the tool system failing?



Versioning & Deprecation

Testing and Production

A safe process:



Example

`create_calendar_event_v1`



`create_calendar_event_v2`



Production Readiness Checklist

Testing and Production

Before deploying a tool, ask:

Design

- Is its responsibility clear?
- Is the name descriptive?
- Are inputs and outputs defined?

Safety

- Are permissions defined?
- Are approval requirements clear?

Reliability

- Are errors defined?
- Is recovery tested?

Observability

- Can we monitor failures, latency, retries, and cost?

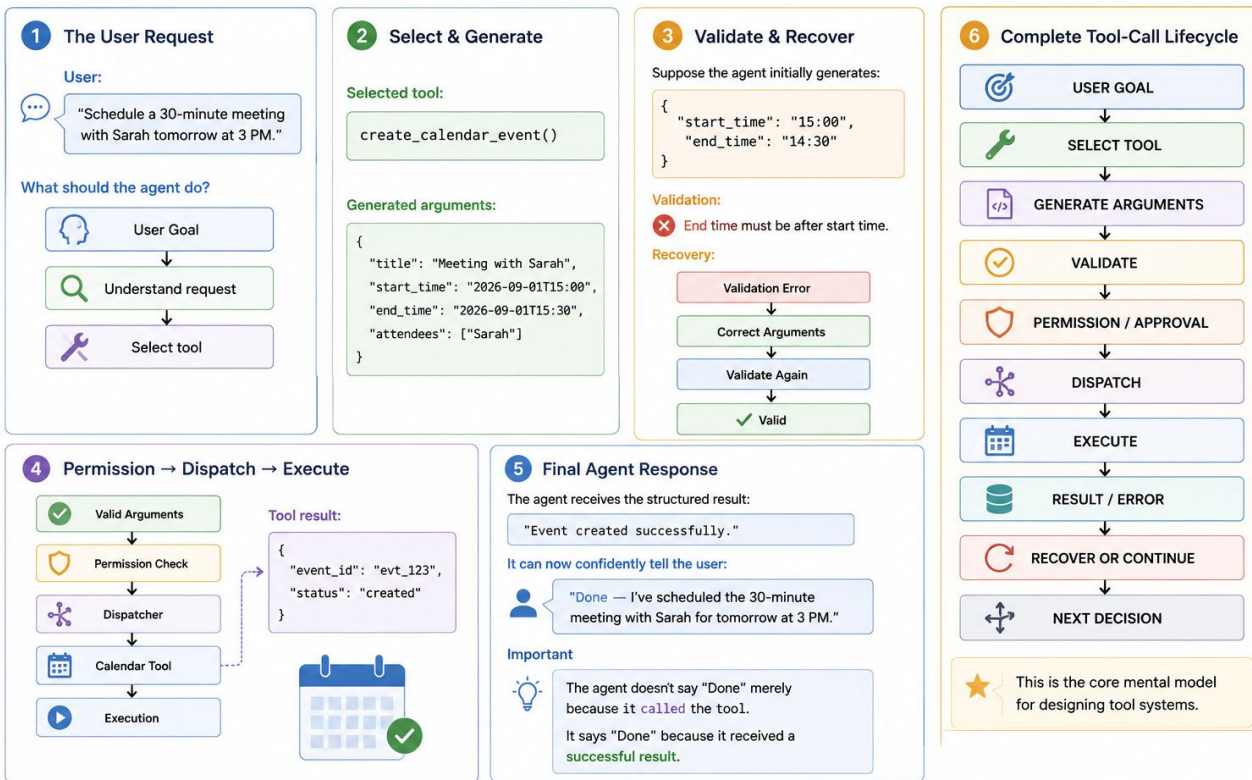
Maintenance

- Can the tool be versioned safely?



End to End Example

Testing and Production





Summary

- **Clear contracts** help agents choose and use tools correctly.
- **Structured inputs** make tool calls predictable.
- **Validation** catches invalid arguments before execution.
- **Error contracts + recovery** help agents handle failures.
- **Registries, dispatchers, and orchestrators** make tool systems manageable.
- **Testing and observability** reveal whether tools actually work in production.
- **Permissions and approvals** keep tool use controlled.



WeCloudData

Thank You

We'd love to hear from you. If you have any questions,
feel free to reach out to our team anytime!